After a successful match using either =~ or match, but *not* using match?, Ruby sets a whole bunch of magic global variables with data.

- The global variable $~ receives the entire MatchData object
- The same MatchData object is also accessible as Regexp.last_match.
- $& receives the complete matched text
- $` receives the part of the string that preceded the match
- $' receives the string after the match.
- $1 receives the first capture group, $2 the second, and so on. More on that in Grouping, on page 138

However, these particular variables are considered to be fairly ugly, so most Ruby programmers instead use the MatchData object returned from the match method, because it encapsulates all the information Ruby knows about the match. Given a MatchData object, you can call pre_match to return the part of the string before the match, post_match for the string after the match, and index using [0] to get the matched portion.

```
"Faster than a speeding bullet" =~ /speed/ # => 14
$~                                         # => #<MatchData "speed">
$&                                         # => "speed"
$`                                         # => "Faster than a "
$'                                         # => "ing bullet"

match_data = "Faster than a speeding bullet".match(/speed/)
match_data                                 # => #<MatchData "speed">
match_data[0]                              # => "speed"
match_data.pre_match                       # => "Faster than a "
match_data.post_match                      # => "ing bullet"
```

We can use these to write show_regexp, a method that shows where a pattern matches:

tut_regexp/show_match.rb

```
def show_regexp(string, pattern)
  match = pattern.match(string)
  if match
    "#{match.pre_match}->#{match[0]}<-#{match.post_match}"
  else
    "no match"
  end
end
```

We could use this method like this:

```
show_regexp('very interesting', /t/) # => very in->t<-eresting
show_regexp('Fats Waller', /lle/)    # => Fats Wa->lle<-r
show_regexp('Fats Waller', /z/)      # => no match
```

Regular Expression Syntax

We said earlier that, within a pattern, all characters match themselves except . | ( ) [ ] { } + \ ^ $ * and ?. Those characters all have special meanings in regular expression patterns. First, always remember that you need to escape any of these characters with a backslash if you want them to be treated as regular characters to match:

```
show_regexp('yes | no', /\|/)       # => yes ->|<- no
show_regexp('yes (no)', /\(no\)/)   # => yes ->(no)<-
```